

Food Delivery Platform

Low-Level Design Document

Final Aligned | Version 2.0

Prepared for System Design Discussion

Authors	Rishabh Jain, Priyanshu Kumar
Version	2.0 — Final Aligned
Scale	50M users · 1M restaurants · 2M DAU · 1M drivers
Architecture	Microservices · PostgreSQL 15+ · PostGIS · Redis · Kafka

1. Overview

This document describes the system design for a scalable food delivery platform comparable to leading marketplace applications. It defines required behavior and the performance qualities required to support large-scale operation across users, restaurants, and delivery partners.

2. Functional Requirements

2.1 User-Side Features

- User Registration — phone-OTP primary; email/OAuth optional.
- User Authentication — secure login with OTP.
- Restaurant Discovery — browse restaurants serviceable in the user's selected location.
- Menu Viewing — items, images, prices, and availability per restaurant.
- Cart Management — restaurant-specific cart; switching restaurants prompts cart clear.
- Place Order — with payment integration.
- Order History — view previous orders; reorder (V2).
- Real-Time Order Tracking — live driver location and order status.
- Order Cancellation (V2) — allowed before preparation begins.
- Address Management — add, edit, and save multiple delivery addresses.
- Ratings & Reviews (V2).
- Search (V2) — restaurants and dishes by keyword.
- Notifications — push/SMS/email at each order stage.
- First-time User Onboarding.

2.2 Restaurant-Side Features

- Restaurant Registration & Owner KYC (FSSAI, GSTIN, PAN, bank).
- Authentication.
- Order Management — accept or reject incoming orders.
- Menu Management — CRUD on items with images.
- Restaurant Onboarding — per-restaurant document set, serviceable zones.
- Availability Management — item-level (manual or time-based).
- Order Status Updates.

2.3 Delivery Partner Features

- Registration & KYC (DL, vehicle, insurance, bank).
- Authentication.
- Order Assignment — push model with timeout/accept/reject.
- Availability toggle + continuous GPS streaming.
- Delivery Status Updates: assigned → en_route_to_restaurant → arrived_at_restaurant → picked_up → out_for_delivery → delivered.
- Earnings & History.

3. Non-Functional Requirements

Requirement	Details
Scalability	50M registered users, 1M restaurants, 2M DAU, 1M registered delivery partners. Horizontal scaling for peak meal-hour spikes.
High Availability	99.95% on read paths (discovery, menu); RTO ≤ 30 min, RPO ≤ 5 min. Redundancy, failover, distributed infra.
Strong Consistency	Payment and inventory require strict consistency. No charge without confirmed order; unavailable items cannot be purchased.
Performance	Discovery/menu p95 ≤ 300 ms; tracking p95 ≤ 200 ms.
Reliability	Exactly-once order semantics via idempotency keys.
Security	TLS 1.2+ in transit; AES-256-GCM (KMS) at rest; PCI-DSS for payment paths; secure authN, authZ.
Observability	Structured logs, RED/USE metrics, W3C trace context, SLO alerts.
Disaster Recovery	Cross-AZ replication; daily logical + 7-day PITR backups; quarterly DR drill.

4. Core Domain Model

4.1 Core Entities

Entity	Responsibility
User	The customer or operator account holder; one of four roles.
Restaurant	Manages food items, menus, and order fulfilment.
Delivery Partner	Picks up and delivers orders to the user.
Order	Central entity connecting users, restaurants, and delivery partners.

4.2 Supporting Entities

4.2.1 Food & Menu Management

- Menu — collection of food items offered by a restaurant.
- MenuCategory — grouping of items (Starters, Mains, etc.).
- MenuItem — individual food items available for ordering.

4.2.2 Ordering Flow

- Cart — temporary storage for items selected before placing an order.
- OrderItem — individual items within a placed order (price-frozen snapshot).

4.2.3 Payment

- Payment — transaction details and processing records.
- Refund — separate lifecycle for post-capture refunds.

4.2.4 Location & Delivery

- Address — user delivery locations.
- UserLocation — last-known resolved location (serviceability).
- Delivery / DeliveryRequest — assignment + active fulfilment.
- DriverLocationLog — high-frequency GPS history.

4.2.5 Identity & Audit

- Onboarding / OnboardingDocument.
- BankAccount.
- Session / OtpRequest.
- Notification.
- AuditLog (append-only).

5. API Design

Microservices architecture. All endpoints are versioned under `/api/v1/*`. Protected endpoints require Authorization: Bearer `<jwt>`; the JWT's `jti` is cross-checked against Redis `session:{sid}`. POST `/orders` and POST `/payments/initiate` require an Idempotency-Key header.

Standard error envelope:

```
{"status":"error","error_code":"STR","message":"STR","details":[]}
```

Auth/role legend used throughout this section:

Code	Auth	Roles
PUB	None	—
AUTH	JWT	any
C	JWT	client
RO	JWT	restaurant_owner
RM	JWT	restaurant_manager
OWN	JWT	restaurant_owner OR restaurant_manager
D	JWT	driver

5.1 User Service

Handles user registration, authentication, profile management, onboarding, and delivery address management.

POST `/api/v1/auth/check-phone` — PUB | 10 req/min/IP

Routes user to register vs. send-OTP based on phone presence.

Request Body:

```
{
  "phone": "9876543210",
  "role": "client"
}
```

Success Response — Case 1 (registered):

```
{
  "status": "success",
  "data": {
    "registered": true,
    "account_status": "active",
    "message": "Proceed to OTP"
  }
}
```

Success Response — Case 2 (suspended):

```
{
  "status": "error",
  "error_code": "ACCOUNT_SUSPENDED",
  "data": {
    "registered": true,
    "account_status": "suspended",
    "message": "Your account has been suspended. Please contact support."
  }
}
```

```
}
```

Errors: INVALID_PHONE (400), RATE_LIMIT_EXCEEDED (429)

POST **/api/v1/auth/register** — PUB | 5 req/hour/phone

Called only for first-time users. Creates the account and sends an OTP.

Request Body:

```
{
  "phone": "9876543210",
  "name": "Rishabh Jain",
  "email": "rishabh@example.com",
  "referral_code": "",
  "role": "client"
}
```

Success Response:

```
{
  "status": "success",
  "data": {
    "otp_sent": true,
    "otp_expires_at": "2024-01-15T10:35:00Z",
    "masked_phone": "+91XXXXXX3210",
    "message": "OTP sent to registered phone number and valid for 10 minutes"
  }
}
```

Errors: PHONE_ALREADY_REGISTERED (409), INVALID_EMAIL (400), RATE_LIMIT_EXCEEDED (429)

POST **/api/v1/auth/send-otp** — PUB | 5/hour/phone

Explicitly resend or request OTP for an existing user.

```
{"phone": "9876543210"}
```

Success Response: same shape as 5.1.2.

Errors: USER_NOT_FOUND (404), RATE_LIMIT_EXCEEDED (429)

POST **/api/v1/auth/verify-otp** — PUB | 5 attempts per OTP

Common to both signup and login. Returns auth tokens and full user profile on success.

Request Body:

```
{"phone": "9876543210", "otp": "439954"}
```

Success Response:

```
{
  "status": "success",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refresh_token": "eyJ...",
    "token_type": "Bearer",
    "expires_in": 900,
    "user": {
      "user_id": "98076550",
      "name": "Rishabh",
      "email": "rishabh@example.com",
      "phone": "9876543210",
      "email_verified": true,
      "phone_verified": true,
      "referral_code": "",
      "account_status": "active",
      "registered_at": "2021-11-15T21:47:36Z",
      "addresses": [],

```

```
    "role": "client",
    "first_time_user": true
  }
}
```

Errors: OTP_INVALID (401), OTP_EXPIRED (410), OTP_MAX_ATTEMPTS (429)

POST **/api/v1/auth/logout** — AUTH

```
{}
```

```
{"status": "success", "data": {"message": "Logged out successfully"}}
```

POST **/api/v1/onboarding/{role}/init** — AUTH (role-matched)

Initializes onboarding. Bodies vary by role; responses include onboarding_id, onboarding_status (initially draft), and required_uploads with pre-signed S3 PUT URLs (5 min TTL).

Case 1 — Client Request Body:

```
{
  "date_of_birth": "1995-06-15",
  "gender": "male",
  "default_address": {
    "line1": "Flat 4B, Shivaji Nagar",
    "line2": "",
    "area": "Civil Lines",
    "city": "Wardha",
    "state": "Maharashtra",
    "pincode": "442001",
    "latitude": 20.745,
    "longitude": 78.602
  }
}
```

Case 1 — Success Response:

```
{
  "status": "success",
  "data": {
    "onboarding_id": "onb_c2a9f1e3",
    "onboarding_status": "draft",
    "required_uploads": [
      {"document_type": "profile_photo", "s3_url": "https://s3.amazonaws.com/..."}
    ]
  }
}
```

Case 2 — Restaurant Owner Request Body:

```
{
  "fssai_license_number": "12345678901234",
  "fssai_expiry_date": "2026-12-31",
  "gst_in": "27AAPFU0939F1ZV",
  "pan_number": "AAPFU0939F",
  "bank_account": {
    "account_number": "XXXXXXXXXXXXXXXX",
    "ifsc_code": "SBIN0001234",
    "account_holder_name": "Ramesh Verma"
  }
}
```

Case 2 — Success Response:

```
{
```

```

"status": "success",
"data": {
  "onboarding_id": "onb_r7k2m4p9",
  "onboarding_status": "draft",
  "owner": {
    "user_id": "98076551",
    "name": "Ramesh Verma",
    "email": "ramesh@example.com",
    "phone": "9876543210",
    "email_verified": true,
    "phone_verified": true
  },
  "legal": {
    "fssai_license_number": "123456XXXXXXX",
    "fssai_expiry_date": "2026-12-31",
    "gst_in": "27AAPFXXXXX",
    "pan_number": "AAPFXXXXX",
    "bank_account": {
      "account_number_masked": "XXXXXXXXXXXX3456",
      "ifsc_code": "SBIN0001234",
      "account_holder_name": "Ramesh Verma"
    }
  },
  "required_uploads": [
    { "document_type": "fssai_license", "s3_url": "https://s3.amazonaws.com/uploads/onb_r7k2m4p9/fssai_license" },
    { "document_type": "owner_profile_photo", "s3_url": "https://s3.amazonaws.com/uploads/onb_r7k2m4p9/owner_profile_photo" }
  ],
  "next_step": {
    "action": "add_restaurant",
    "message": "Once your documents are approved, add your first restaurant.",
    "endpoint": "POST /api/v1/restaurants"
  }
}
}

```

Case 3 — Driver Request Body (vehicle.type ∈ bike | scooter | car):

```

{
  "driving_license": {
    "number": "MH3720230012345",
    "expiry_date": "2033-08-20"
  },
  "vehicle": {
    "type": "bike",
    "registration_number": "MH12AB1234",
    "make": "Honda",
    "model": "Activa 6G",
    "year": 2022,
    "insurance_expiry_date": "2025-06-30"
  },
  "bank_account": {
    "account_number": "XXXXXXXXXXXXXXXXX",
    "ifsc_code": "SBIN0001234",
    "account_holder_name": "Rishabh Jain"
  }
}

```

Case 3 — Success Response:

```
{
  "status": "success",
  "data": {
    "onboarding_id": "onb_d7f3a291",
    "onboarding_status": "draft",
    "required_uploads": [
      { "document_type": "profile_photo", "s3_url": "https://s3.amazonaws.com/..." },
      { "document_type": "driving_license", "s3_url": "https://s3.amazonaws.com/..." },
      { "document_type": "rc_book", "s3_url": "https://s3.amazonaws.com/..." },
      { "document_type": "insurance", "s3_url": "https://s3.amazonaws.com/..." }
    ]
  }
}
```

Errors: ONBOARDING_ALREADY_EXISTS (409), INVALID_KYC_FORMAT (400), FSSAI_EXPIRED (400)

GET </api/v1/onboarding/{role}/status> — AUTH

```
{
  "status": "success",
  "data": {
    "onboarding_id": "onb_d7f3a291",
    "onboarding_status": "rejected",
    "rejection_reasons": [
      { "field": "driving_license", "reason": "Document is blurry or unreadable" },
      { "field": "insurance", "reason": "Document appears expired" }
    ],
    "verified_at": null,
    "submitted_at": "2024-01-15T10:50:00Z"
  }
}
```

PATCH </api/v1/onboarding/{onboardingId}/submit> — AUTH

Transitions draft → pending_verification once all required_uploads are in 'uploaded' state.

```
{
  "status": "success",
  "data": {
    "onboarding_id": "onb_d7f3a291",
    "onboarding_status": "pending_verification"
  }
}
```

Errors: UPLOADS_INCOMPLETE (412), ONBOARDING_NOT_FOUND (404)

POST </api/v1/onboarding/{onboardingId}/resubmit> — AUTH

Resubmit only the rejected fields. Returns fresh pre-signed URLs and resets status to pending_verification.

Request Body:

```
{ "resubmit_fields": [ "driving_license", "insurance" ] }
```

Success Response:

```
{
  "status": "success",
  "data": {
    "onboarding_id": "onb_d7f3a291",
    "required_uploads": [
      { "document_type": "driving_license", "s3_url": "https://s3.amazonaws.com/..." },
      { "document_type": "insurance", "s3_url": "https://s3.amazonaws.com/..." }
    ]
  }
}
```

```
}  
}
```

GET

</api/v1/users/me/profile> — AUTH

Single call on app open / profile screen — returns role-shaped profile. Sensitive fields are masked. addresses[] uses line1, line2, area, city, state, pincode, latitude, longitude.

Case 1 — role: client:

```
{
  "status": "success",
  "data": {
    "role": "client",
    "identity": {
      "user_id": "98076550",
      "name": "Rishabh",
      "email": "rishabh@example.com",
      "phone": "7909338983",
      "email_verified": true,
      "phone_verified": true,
      "profile_image": "https://cdn.example.com/avatars/usr_3f2a1b4c.jpg",
      "referral_code": "",
      "registered_at": "2021-11-15T21:47:36Z"
    },
    "profile": { "date_of_birth": "1995-06-15", "gender": "male" },
    "addresses": {
      "items": [
        {
          "address_id": "d4ljnrg4ce1o6mo90j20",
          "label": "Home",
          "line1": "205 Stanza Living",
          "line2": "",
          "area": "Koramangala",
          "city": "Bangalore",
          "state": "Karnataka",
          "pincode": "560029",
          "latitude": 12.9345,
          "longitude": 77.6089,
          "contact_name": "Rishabh",
          "contact_phone": "7909338983",
          "created_at": "2025-11-30T00:01:10Z"
        }
      ],
      "count": 1,
      "limit": 20
    }
  }
}
```

Case 2 — role: restaurant_owner:

```
{
  "status": "success",
  "data": {
    "role": "restaurant_owner",
    "identity": {
      "user_id": "98076551", "name": "Ramesh Verma", "email": "ramesh@example.com",
      "phone": "9876543210", "email_verified": true, "phone_verified": true,
      "profile_image": "https://cdn.example.com/avatars/usr_r1a2b3c4.jpg",
      "registered_at": "2024-03-01T10:00:00Z"
    },
    "profile": {
      "onboarding_status": "approved",
    }
  }
}
```

```
{
  "documents":{
    "fssai_license_number":"123456XXXXXXX", "fssai_expiry_date":"2026-12-31",
    "gstn":"27AAPFUXXXX", "pan_number":"AAPFUXXXX"
  },
  "bank_account":{
    "account_number_masked":"XXXXXXXXXX3456",
    "ifsc_code":"SBIN0001234",
    "account_holder_name":"Ramesh Verma"
  }
},
"restaurants":{
  "items":[
    {
      "restaurant_id":"rst_4a1b2c3d",
      "restaurant_manager_name":"Aashish",
      "restaurant_manager_phone":"9914173314",
      "restaurant_name":"Ramesh's Dhaba",
      "cuisine_types":["north_indian","street_food"],
      "status":"active",
      "address":{
        "line1":"Shop 4, MG Road","line2":"","area":"Civil Lines",
        "city":"Wardha","state":"Maharashtra","pincode":"442001",
        "latitude":20.745,"longitude":78.602
      },
      "thumbnail":"https://cdn.example.com/restaurants/rst_4a1b2c3d/thumb.jpg",
      "created_at":"2024-03-01T10:00:00Z"
    }
  ],
  "count":1
}
}
```

Case 3 — role: driver:

```
{
  "status":"success",
  "data":{
    "role":"driver",
    "identity":{
      "user_id":"98076552","name":"Rishabh Jain","email":"rj@example.com",
      "phone":"7909338983","email_verified":true,"phone_verified":true,
      "profile_image":"https://s3.example.com/avatars/usr_d9x8y7z6.jpg",
      "registered_at":"2024-06-15T08:30:00Z"
    },
    "profile":{
      "onboarding_status":"approved",
      "driving_license":{"number_masked":"MH37XXXXXX2345","expiry_date":"2033-08-20"},
      "vehicle":{
        "type":"bike",
        "registration_number_masked":"MH12XXXXX234",
        "make":"Honda","model":"Activa 6G","year":2022,
        "insurance_expiry_date":"2025-06-30"
      },
      "bank_account":{
        "account_number_masked":"XXXXXXXXXX7890",
        "ifsc_code":"SBIN0001234",
        "account_holder_name":"Rishabh Jain"
      }
    }
  }
}
```

```
}  
}
```

PUT **/api/v1/users/me/profile** — AUTH

Changing email resets email_verified=false and dispatches a verification email.

Case 1 — client:

```
{  
  "identity":{"name":"Rishabh Kumar","email":"new@example.com"},  
  "profile":{"date_of_birth":"1995-06-15","gender":"other"}  
}  
  
{  
  "status":"success",  
  "data":{  
    "role":"client",  
    "identity":{  
      "user_id":"98076550","name":"Rishabh Kumar","email":"new@example.com",  
      "email_verified":false,"phone_verified":true,  
      "updated_at":"2026-03-31T10:00:00Z"  
    },  
    "profile":{"date_of_birth":"1995-06-15","gender":"other"},  
    "warnings":[  
      {"code":"EMAIL_VERIFICATION_REQUIRED","message":"A verification link has been sent to new@exampl  
e.com."}  
    ]  
  }  
}
```

Case 2 — restaurant_owner (analogous shape; restaurants[] updates limited to allowed fields). Case 3 — driver (analogous; vehicle make/model/year are user-mutable).

GET **/api/v1/users/me/addresses** — AUTH

```
{  
  "addresses":{  
    "items":[  
      {  
        "address_id":"d4ljnrg4ce1o6mo90j20",  
        "label":"Home",  
        "line1":"205 Stanza Living",  
        "line2":"","  
        "area":"Koramangala",  
        "city":"Bangalore",  
        "state":"Karnataka",  
        "pincode":"560029",  
        "latitude":12.9345,  
        "longitude":77.6089,  
        "contact_name":"Rishabh",  
        "contact_phone":"7909338983",  
        "created_at":"2025-11-30T00:01:10Z"  
      }  
    ]  
  }  
}
```

POST **/api/v1/users/me/addresses** — AUTH

Path no longer carries {userId}; identity is taken from the JWT (closes IDOR).

```
{
```

```
"label":"Home",
"line1":"42 MG Road",
"line2":"Flat 3B",
"area":"Brigade Road",
"city":"Bengaluru",
"state":"Karnataka",
"pincode":"560001",
"latitude":12.9716,
"longitude":77.5946,
"contact_name":"Rishabh",
"contact_phone":"7909338983"
}
```

201 Created:

```
{
  "address_id":"addr_9d1e2f3a",
  "label":"Home",
  "line1":"42 MG Road",
  "city":"Bengaluru",
  "created_at":"2024-03-20T08:15:00Z"
}
```

PUT**/api/v1/users/me/addresses/{addressId}** — AUTH

```
{
  "label":"Work",
  "line1":"10 Residency Road",
  "line2":"",
  "area":"Ashok Nagar",
  "city":"Bengaluru",
  "state":"Karnataka",
  "pincode":"560001",
  "latitude":12.9716,
  "longitude":77.5946,
  "contact_name":"Rishabh",
  "contact_phone":"7909338983"
}
```

```
{
  "address_id":"addr_9d1e2f3a",
  "label":"Work",
  "line1":"10 Residency Road",
  "city":"Bengaluru",
  "updated_at":"2024-03-21T09:00:00Z"
}
```

DELETE**/api/v1/users/me/addresses/{addressId}** — AUTH (soft delete)

```
{"message":"Address deleted successfully","address_id":"addr_9d1e2f3a"}
```

5.2 Restaurant Service

Manages restaurant onboarding, profile, menu, and location-based discovery.

GET

/api/v1/restaurants — PUB | 60/min/IP

Query: latitude, longitude, radius_km (1..25), cuisine, sort_by, page, limit (≤50), cursor.

```
GET /api/v1/restaurants?latitude=12.9716&longitude=77.5946&radius_km=5&cuisine=Indian&sort_by=rating&page=1&limit=20
```

```
{
  "restaurants": [
    {
      "restaurant_id": "rst_a1b2c3d4",
      "restaurant_profile_image": "https://s3.amazonaws.com/bucket/spice-garden.jpg",
      "name": "Spice Garden",
      "cuisine_types": ["Indian", "Biryani"],
      "rating": 4.3,
      "city": "Bengaluru",
      "distance_km": 1.2,
      "delivery_time_min": 35,
      "opening_cron": "0 9 * * 1-6",
      "is_open": true
    }
  ],
  "total": 48,
  "page": 1,
  "limit": 20,
  "next_cursor": "eyJvZmZzZXQiOjIwIiwfQ=="
}
```

GET

/api/v1/restaurants/{restaurantId} — PUB

```
{
  "restaurant_id": "rst_a1b2c3d4",
  "name": "Spice Garden",
  "restaurant_profile_image": "https://s3.amazonaws.com/bucket/spice-garden.jpg",
  "cuisine_types": ["Indian", "Biryani"],
  "rating": 4.3,
  "address": {
    "line1": "15 Brigade Road",
    "city": "Bengaluru",
    "latitude": 12.9752,
    "longitude": 77.6073
  },
  "distance_km": 1.2,
  "opening_cron": "0 9 * * 1-6",
  "is_open": true,
  "delivery_time_min": 35
}
```

GET

/api/v1/restaurants/{restaurantId}/menu — PUB

```
GET /api/v1/restaurants/{restaurantId}/menu?page=1&limit=50&category=Starters
```

```
{
  "restaurant_id": "rst_a1b2c3d4",
  "categories": [
    {
```

```
    "category": "Starters",
    "items": [
      {
        "item_id": "itm_x1y2z3",
        "name": "Paneer Tikka",
        "price": 220.00,
        "is_veg": true,
        "is_available": true,
        "description": "Soft paneer marinated and grilled in a tandoor",
        "image_url": "https://s3.amazonaws.com/bucket/paneer_tikka.jpg"
      }
    ]
  }
},
"total_items": 300,
"page": 1,
"limit": 50
}
```

POST /api/v1/restaurants — RO

```
{
  "name": "Spice Garden",
  "cuisine_types": ["Indian", "Biryani"],
  "address": {
    "line1": "15 Brigade Road",
    "city": "Bengaluru",
    "state": "Karnataka",
    "pincode": "560001",
    "latitude": 12.9752,
    "longitude": 77.6073
  },
  "service_radius_km": 8.0,
  "opening_cron": "0 9 * * 1-5",
  "compliance": {
    "fssai_license": "100200420000001",
    "gst_in": "29ABCDE1234F1Z5"
  },
  "logo_url": "restaurants/logos/rst_a1b2c3d4/logo.png"
}
```

201 Created:

```
{
  "restaurant_id": "rst_a1b2c3d4",
  "name": "Spice Garden",
  "status": "pending_review",
  "created_at": "2024-03-20T08:15:00Z"
}
```

Errors: KYC_NOT_APPROVED (403), OWNER_HAS_PENDING_RESTAURANT (409)

PUT /api/v1/restaurants/{restaurantId} — OWN

Allowed fields: name, cuisine_types, service_radius_km, opening_cron, logo_url. Disallowed fields (compliance, address, status) return 400 FIELD_NOT_ALLOWED.

```
{
  "name": "Spice Garden Premium",
  "cuisine_types": ["Indian", "Biryani", "Mughlai"],
  "service_radius_km": 10.0,
  "opening_cron": "0 9 * * *",
}
```

```
"logo_url":"restaurants/logos/rst_a1b2c3d4/logo_v2.png"
}
```

```
{
  "restaurant_id":"rst_a1b2c3d4",
  "name":"Spice Garden Premium",
  "status":"active",
  "updated_at":"2024-04-01T11:00:00Z"
}
```

POST

/api/v1/restaurants/{restaurantId}/menu — OWN

```
{
  "categories":[
    {
      "name":"Starters",
      "items":[
        {
          "name":"Paneer Tikka",
          "description":"Grilled cottage cheese with spices",
          "price":220.00,
          "image_url":"restaurants/menu/rst_a1b2c3d4/paneer_tikka.png",
          "is_veg":true,
          "is_available":true,
          "preparation_time_min":15,
          "tags":["bestseller","spicy"]
        }
      ]
    }
  ]
}
```

201 Created:

```
{
  "restaurant_id":"rst_a1b2c3d4",
  "menu_id":"mnu_p9q8r7s6",
  "total_items":12,
  "items_created":11,
  "items_not_created":1,
  "items":[
    {"item_id":"itm_x1y2z3","name":"Paneer Tikka","category":"Starters","status":"created"}
  ],
  "created_at":"2024-03-20T08:15:00Z"
}
```

POST

/api/v1/restaurants/{restaurantId}/menu/items — OWN

```
[
  {
    "name":"Paneer Tikka",
    "description":"Grilled cottage cheese with spices",
    "category":"Starters",
    "price":220.00,
    "image_url":"s3_url",
    "is_veg":true,
    "is_available":true,
    "preparation_time_min":15,
    "tags":["bestseller","spicy"]
  }
]
```

201 Created:

```
{
  "item_id": "itm_x1y2z3",
  "name": "Paneer Tikka",
  "price": 220.00,
  "is_available": true,
  "created_at": "2024-03-20T08:15:00Z"
}
```

PUT

/api/v1/restaurants/{restaurantId}/menu/items/{itemId} — OWN

```
{
  "price": 240.00,
  "is_available": false,
  "description": "Grilled cottage cheese with house spice blend",
  "tags": ["bestseller", "spicy", "new"]
}
```

```
{
  "restaurant_id": "rst_a1b2c3d4",
  "item_id": "itm_x1y2z3",
  "name": "Paneer Tikka",
  "price": 240.00,
  "is_available": false,
  "updated_at": "2024-04-01T11:00:00Z"
}
```

DELETE

/api/v1/restaurants/{restaurantId}/menu/items/{itemId} — OWN (soft delete)

// 204 No Content

5.3 Cart Service

Cart token format (external, opaque): `cart.<base64(cart_id)>.<hmac_sha256(cart_id, secret)>`. Server validates HMAC and resolves to internal `cart_id` (UUID).

POST `/api/v1/cart` — C | 30/min/user

```
{
  "cart_token": "cart.dXNlcjEyMz.hmac_signature",
  "restaurant_id": "rst_a1b2c3d4",
  "item_id": "itm_x1y2z3",
  "quantity": 2,
  "customisations": ["Extra spicy", "No onion"]
}
```

If `cart_token` is missing or HMAC-invalid, server creates a fresh cart and returns a new token.

```
{
  "status": "success",
  "data": {
    "cart_token": "cart.dXNlcjEyMz.hmac_signature",
    "cart_item_id": "ci_abc123",
    "message": "Item added to cart"
  }
}
```

Errors: RESTAURANT_MISMATCH (409), ITEM_UNAVAILABLE (409), CART_TOKEN_INVALID (401)

GET `/api/v1/cart/{cartToken}` — C

Response excludes `delivery_fee` and `total` — those are computed at order/quote/place (they depend on the chosen delivery address).

```
{
  "cart_token": "cart.dXNlcjEyMz.hmac_signature",
  "restaurant_id": "rst_a1b2c3d4",
  "restaurant_name": "Spice Garden",
  "items": [
    {
      "cart_item_id": "ci_abc123",
      "item_id": "itm_x1y2z3",
      "name": "Paneer Tikka",
      "quantity": 2,
      "unit_price": 220.00,
      "customisations": []
    },
    {
      "cart_item_id": "ci_def456",
      "item_id": "itm_x1y2z3",
      "name": "Paneer Tikka",
      "quantity": 1,
      "unit_price": 220.00,
      "customisations": ["Extra spicy"]
    },
    {
      "cart_item_id": "ci_ghi789",
      "item_id": "itm_x1y2z3",
      "name": "Paneer Tikka",
      "quantity": 1,
      "unit_price": 220.00,
      "customisations": ["Less spicy"]
    }
  ],
  "subtotal": 890.00
}
```

DELETE `/api/v1/cart/{cartToken}/items/{cartItemId}` — C

```
{
  "cart_item_id": "ci_abc123",
  "message": "Cart item cleared successfully",
  "success": true
}
```

DELETE `/api/v1/cart/{cartToken}` — C

```
{
  "cart_token": "cart.dXNlcjEyMz.hmac_signature",
}
```

```
"message":"All items deleted from cart successfully",
"success":true
}
```

5.4 Order Service

Cart finalization, order placement, real-time tracking, and order status management.

POST

/api/v1/orders/quote — C

Server-side pricing pre-placement. No DB write.

```
{"cart_token":"cart.dXNlcjEyMz.hmac_signature","address_id":"addr_9d1e2f3a"}
```

```
{
  "subtotal":890.00,
  "taxes":53.40,
  "delivery_fee":30.00,
  "discount":0.00,
  "total_amount":973.40,
  "estimated_delivery_at":"2024-03-20T09:05:00Z"
}
```

POST

/api/v1/orders — C, idempotency required

Header: Idempotency-Key: <uuid>

```
{
  "cart_token":"cart.dXNlcjEyMz.hmac_signature",
  "address_id":"addr_9d1e2f3a",
  "payment_method":"upi",
  "instructions":"Please ring the bell twice"
}
```

201 Created:

```
{
  "order_id":"ord_m1n2o3p4",
  "status":"order_created",
  "total_amount":973.40,
  "estimated_delivery":"2024-03-20T09:05:00Z",
  "created_at":"2024-03-20T08:30:00Z"
}
```

Errors: CART_EMPTY (400), ITEM_PRICE_CHANGED (409), ADDRESS_NOT_SERVICEABLE (422), IDEMPOTENCY_REPLAY (200)

GET

/api/v1/orders/{orderId} — C / OWN / D (scoped)

```
{
  "order_id":"ord_m1n2o3p4",
  "status":"preparing",
  "restaurant":{"restaurant_id":"rst_a1b2c3d4","name":"Spice Garden"},
  "items":[
    {"item_id":"itm_x1y2z3","name":"Paneer Tikka","quantity":2,"price":440.00}
  ],
  "delivery_address":{"line1":"42 MG Road","city":"Bengaluru","pincode":"560001"},
  "total_amount":973.40,
  "created_at":"2024-03-20T08:30:00Z",
  "estimated_delivery":"2024-03-20T09:05:00Z"
}
```

GET

/api/v1/users/me/orders — C

Query: ?page=1&limit=10

```
{
  "orders":[
    {
      "order_id":"ord_m1n2o3p4",
      "restaurant":{"restaurant_id":"rst_a1b2c3d4","name":"Spice Garden"},
      "items":[
        {"item_id":"itm_x1y2z3","name":"Paneer Tikka","quantity":2,"price":440.00}
      ],
      "delivery_address":{"line1":"42 MG Road","city":"Bengaluru","pincode":"560001"},
      "status":"delivered",
      "total_amount":973.40,
      "created_at":"2024-03-20T08:30:00Z"
    }
  ],
  "total":15
}
```

POST [/api/v1/orders/{orderId}/cancel](#) — C

Allowed only when status ∈ {order_created, pending_payment, confirmed}.

```
{"reason":"Changed my mind"}
```

```
{
  "order_id":"ord_m1n2o3p4",
  "status":"cancelled",
  "refund_initiated":true,
  "cancelled_at":"2024-03-20T08:35:00Z"
}
```

PUT [/api/v1/orders/{orderId}/status](#) — OWN / D (state-machine enforced)

```
{
  "status":"preparing",
  "updated_by":"rst_a1b2c3d4",
  "notes":"Will be ready in 20 minutes"
}
```

```
{
  "order_id":"ord_m1n2o3p4",
  "status":"preparing",
  "updated_at":"2024-03-20T08:35:00Z"
}
```

Allowed transitions:

```
order_created -> pending_payment -> confirmed -> accepted -> preparing
               -> ready_for_pickup -> picked_up -> out_for_delivery -> delivered
               \-> cancelled (pre-preparing) | rejected (by restaurant pre-accepted)
```

GET [/api/v1/orders/{orderId}/tracking](#) — C (owner only)

```
{
  "order_id":"ord_m1n2o3p4",
  "status":"picked_up",
  "driver":{"
    "name":"Arjun Kumar",
    "phone_masked":"+9198XXXXXX78",
    "vehicle":"KA 05 MX 1234"
  },
  "driver_location":{"
```

```
    "latitude":12.9730,"longitude":77.5985,  
    "updated_at":"2024-03-20T08:55:00Z"  
  },  
  "eta_minutes":8,  
  "status_timeline":[  
    {"status":"order_created","timestamp":"2024-03-20T08:30:00Z"},  
    {"status":"confirmed","timestamp":"2024-03-20T08:32:00Z"},  
    {"status":"picked_up","timestamp":"2024-03-20T08:52:00Z"}  
  ]  
}
```

5.5 Payment Service

POST

/api/v1/payments/initiate — C, idempotency required

```
{
  "order_id": "ord_m1n2o3p4",
  "payment_method": "upi",
  "payment_token": "tok_rzp_upi_abc123"
}
```

```
{
  "payment_id": "pay_q5r6s7t8",
  "gateway_order_id": "rzp_ord_XYZ789",
  "status": "pending",
  "redirect_url": "https://api.razorpay.com/v1/checkout/embedded",
  "expires_at": "2024-03-20T08:45:00Z"
}
```

Errors: ORDER_ALREADY_PAID (409), GATEWAY_UNAVAILABLE (502)

POST

/api/v1/payments/verify — C

```
{
  "payment_id": "pay_q5r6s7t8",
  "gateway_payment_id": "rzp_pay_ABC123XYZ",
  "gateway_signature": "sha256_hmac_signature_here"
}
```

```
{
  "payment_id": "pay_q5r6s7t8",
  "order_id": "ord_m1n2o3p4",
  "status": "success",
  "amount_paid": 973.40,
  "paid_at": "2024-03-20T08:38:00Z"
}
```

5.6 Notification Service

POST

/api/v1/notifications/send — internal-service-only (mTLS) | 1k/sec

recipient_type ∈ client | restaurant_owner | restaurant_manager | driver. Channels: push | sms | email. priority: integer 1–10.

```
{
  "recipient_id": "usr_3f2a1b4c",
  "recipient_type": "client",
  "channel": ["push", "sms"],
  "title": "Order Confirmed!",
  "body": "Your order from Spice Garden has been confirmed.",
  "data": {
    "order_id": "ord_m1n2o3p4",
    "deep_link": "app://orders/ord_m1n2o3p4"
  },
  "priority": 8
}
```

202 Accepted:

```
{ "notification_id": "ntf_u1v2w3x4", "status": "queued" }
```

GET

/api/v1/notifications/me — AUTH

Query: ?page=1&limit=20&unread_only=false

```
{
  "notifications": [
    {
      "notification_id": "ntf_u1v2w3x4",
      "title": "Order Confirmed!",
      "body": "Your order from Spice Garden has been confirmed.",
      "is_read": false,
      "created_at": "2024-03-20T08:32:00Z"
    }
  ],
  "total": 5,
  "unread_count": 2
}
```

PUT

/api/v1/notifications/{notifId}/read — AUTH

Body is empty; user_id is derived from the JWT (closes IDOR).

```
{}
```

```
{
  "notification_id": "ntf_u1v2w3x4",
  "is_read": true,
  "read_at": "2024-03-20T09:00:00Z"
}
```

Event-Based Notifications (Internal / Async) — triggered automatically by Kafka events:

Kafka Event	Recipient	Notification
order.placed	restaurant_owner / restaurant_manager	New order received with details.
order.confirmed	client	Order confirmed, preparation started.
order.picked_up	client	Delivery partner has picked up your order.
order.delivered	client + driver	Order delivered. Rate your experience.
payment.failed	client	Payment failed. Please retry.

5.7 Delivery Service

POST

/api/v1/delivery/assign — internal (Order Service → Delivery Service)

strategy is VARCHAR(32) whitelisted at the application layer (e.g., nearest, rating_first, surge_aware).

```
{"order_id": "ord_m1n2o3p4", "strategy": "nearest"}
```

```
{
  "request_id": "dreq_x1y2z3a4",
  "order_id": "ord_m1n2o3p4",
  "status": "pending_acceptance",
  "partner_id": "drv_c9d0e1f2",
  "partner_name": "Arjun Kumar",
  "timeout_at": "2024-03-20T08:45:30Z"
}
```

POST

/api/v1/delivery/partner/respond — D | 60/min/driver

```
{
```

```
"request_id":"dreq_x1y2z3a4",
"partner_id":"drv_c9d0e1f2",
"action":"accept"
}
```

Case 1 — accept:

```
{
  "delivery_id":"dlv_y5z6a7b8",
  "order_id":"ord_m1n2o3p4",
  "partner_id":"drv_c9d0e1f2",
  "status":"assigned",
  "estimated_pickup_min":7
}
```

Case 2 — reject/timeout:

```
{
  "request_id":"dreq_x1y2z3a4",
  "status":"rejected",
  "message":"Finding next available partner"
}
```

GET [/api/v1/delivery/{orderId}](#) — C / OWN / D (scoped)

```
{
  "delivery_id":"dlv_y5z6a7b8",
  "order_id":"ord_m1n2o3p4",
  "partner_id":"drv_c9d0e1f2",
  "partner_name":"Arjun Kumar",
  "status":"picked_up",
  "pickup_at":"2024-03-20T08:52:00Z",
  "estimated_delivery_at":"2024-03-20T09:05:00Z"
}
```

PUT [/api/v1/delivery/{orderId}/status](#) — D (state-machine enforced)

Allowed values: assigned | en_route_to_restaurant | arrived_at_restaurant | picked_up | out_for_delivery | delivered.

```
{
  "status":"picked_up",
  "partner_id":"drv_c9d0e1f2",
  "notes":"Left the restaurant on time"
}
```

```
{
  "delivery_id":"dlv_y5z6a7b8",
  "order_id":"ord_m1n2o3p4",
  "status":"picked_up",
  "updated_at":"2024-03-20T08:52:00Z"
}
```

POST [/api/v1/delivery/location](#) — D | 5 req/sec/driver

Anti-spoof: server rejects updates whose implied speed > 120 km/h or jump > 1 km vs prior point unless accuracy_meters > 100.

```
{
  "partner_id":"drv_c9d0e1f2",
  "order_id":"ord_m1n2o3p4",
  "latitude":12.9730,
  "longitude":77.5985,
  "bearing":180.5,
}
```

```
"speed_kmh":28.4
}
```

```
{"recorded":true,"eta_minutes":8}
```

PUT [/api/v1/delivery/partner/availability](#) — D

```
{"is_available":true,"city":"bengaluru"}
```

```
{"status":"success","data":{"is_available":true}}
```

GET [/api/v1/delivery/partner/earnings](#) — D

Query: ?from=2024-03-01&to=2024-03-31&page=1&limit=20

```
{
  "total_earnings":18450.00,
  "total_deliveries":192,
  "items":[
    {"delivery_id":"dlv_y5z6a7b8","order_id":"ord_m1n2o3p4","delivered_at":"2024-03-20T09:05:00Z","pay
out":95.00}
  ],
  "page":1,"limit":20,"total":192
}
```

5.8 Common APIs

POST `/api/v1/users/me/location/resolve` — AUTH

Request Body:

```
{
  "latitude":12.9730,
  "longitude":77.5985,
  "accuracy_meters":18,
  "source":"gps"
}
```

Case 1 — role: client:

```
{
  "status":"success",
  "data":{
    "role":"client",
    "location":{
      "latitude":12.9730,"longitude":77.5985,
      "accuracy_meters":18,"source":"gps",
      "captured_at":"2026-04-01T10:00:00Z"
    },
    "resolved_address":{
      "display":"Koramangala, Bangalore",
      "line1":"4th Block, Koramangala",
      "area":"Koramangala",
      "city":"Bangalore","state":"Karnataka",
      "pincode":"560034","country":"IN"
    },
    "serviceability":{"is_serviceable":true,"message":null},
    "prompt_save_address":true
  }
}
```

Case 2 — role: driver:

```
{
  "status":"success",
  "data":{
    "role":"driver",
    "location":{
      "latitude":12.9730,"longitude":77.5985,
      "accuracy_meters":18,"source":"gps",
      "captured_at":"2026-04-01T10:00:00Z"
    }
  }
}
```

POST `/api/v1/uploads/presigned-url` — AUTH

Server enforces bucket allow-list, owner-scoped path, content-type whitelist, size cap. Pre-signed URL TTL = 5 min, PUT only.

```
{
  "bucket":"ugc-media",
  "path":"users/{user_id}/profile.jpg",
  "content_type":"image/jpeg",
  "max_bytes":5242880
}
```

```
{"presigned_url":"https://s3-url..."}
```

5.9 API Design Considerations

Concern	Approach
Authentication	Short-lived JWT (15 min) with rotating refresh (30 d). JTI cross-checked in session:{sid} (Redis).
Authorization	Role + ownership checks per resource (RBAC + ABAC). All path params {userId} replaced with /me/.
Rate Limiting	Sliding-window per IP and user_id (Redis). Per-endpoint limits documented per API.
Idempotency	Required on POST /orders and POST /payments/initiate. Idempotency-Key UUID; 24 h dedupe in Redis + UNIQUE in DB.
Pagination	Cursor-based; response includes next_cursor and total (where cheap).
Versioning	Prefix /v1/, additive non-breaking. Breaking changes introduce /v2/.
Validation	JSON Schema at gateway; defense-in-depth at service. Reject unknown fields.
Error Handling	{ status, error_code, message, details[] } with deterministic codes per endpoint.
Strong Consistency	DB transactions + Redis distributed lock (SET NX EX) on assignment, payment, inventory.
Observability	Structured JSON logs; PII masked; W3C trace context.
Audit	All write APIs emit a row to audit_logs.

6. Per-Feature Flowcharts

All flowcharts are authored in Mermaid and exported as standalone .svg files in flowcharts/. PNG renders are embedded at each section for review.

Section	Feature	SVG File
6.1	Auth (Check-Phone → OTP → Verify)	auth_otp.svg
6.2	Onboarding	onboarding.svg
6.3	Profile View / Update	profile.svg
6.4	Address CRUD	address_crud.svg
6.5	Restaurant Discovery	restaurant_discovery.svg
6.6	Menu Management	menu_management.svg
6.7	Cart	cart.svg
6.8	Place Order	place_order.svg
6.9	Cancel Order	cancel_order.svg
6.10	Payment	payment.svg
6.11	Driver Assignment	driver_assignment.svg
6.12	Live Tracking & Location Stream	live_tracking.svg
6.13	Notifications	notifications.svg
6.14	Pre-signed Upload	presigned_upload.svg
6.15	Driver Availability + Earnings	driver_availability_earnings.svg

7. Security Considerations

7.1 Per-Feature Security Matrix

Feature	AuthN	AuthZ	Validation	Rate Limit	Other
check-phone	None	—	E.164 phone	10/min/IP	CAPTCHA after 3 failures
register / send-otp	None	—	E.164, name, RFC 5322 email	5/hr/phone	OTP hashed (bcrypt 12)
verify-otp	None	—	6-digit numeric	5 attempts/OTP	Lock account 30 min on 5 fails
logout	JWT	self	—	10/min	Revoke session in Redis
onboarding/*	JWT	role-matched	FSSAI/PAN/GSTIN/IFSC regex	20/min	KYC fields encrypted via KMS
profile get/update	JWT	self	length/format	30/min	Email change resets verified
addresses	JWT	self (path=/me/)	pincode/lat-lng range	60/min	IDOR-proof
restaurants list	None	—	radius ≤ 25 km	60/min/IP	—
restaurant register	JWT	RO	KYC check	5/hr	KYC encrypted
restaurant update	JWT	OWN	field allow-list	30/min	Reject disallowed fields (400)
menu mgmt	JWT	OWN	price > 0, image URL allow-list	60/min	SSRF guard on image_url
cart	JWT	self via HMAC token	cart_token HMAC	30/min	Server-side pricing only
orders/quote, place	JWT	C	Idempotency-Key required (place)	10/min/user	Distributed lock
orders/cancel	JWT	C	status guard	10/min	Refund flow
orders/status	JWT	OWN/D	state machine	60/min	Audit log
tracking	JWT	order-owner	—	60/min	Mask driver phone
payments	JWT	C	Idempotency-Key, HMAC verify	10/min	PCI scope; never log token
notifications send	mTLS	internal service	priority 1–10	1k/sec	Spam guard
notifications me/read	JWT	self	—	60/min	user_id from JWT
delivery assign	mTLS	internal service	strategy whitelist	—	Lock per driver
partner respond	JWT	D	request_id scope	60/min	One-shot
delivery location	JWT	D	anti-spoof	5/sec	Bounded jump check
availability/earnings	JWT	D	from ≤ to	60/min	—

Feature	AuthN	AuthZ	Validation	Rate Limit	Other
location/resolve	JWT	self	lat/lng range	60/min	Geocoder cached
presigned-url	JWT	self	bucket+path+type+size	30/min	Owner-scoped path

7.2 Cross-Cutting Security

- TLS 1.2+ everywhere; HSTS at the edge.
- AES-256-GCM envelope encryption (KMS) for all *_encrypted columns.
- OWASP A03 SQL/NoSQL injection — parameterized queries only; no string concatenation.
- OWASP A05 Security Misconfiguration — secrets via vault; no .env in image; minimal base images.
- OWASP A07 IDOR — every {userId} path replaced by /me/; ownership checked at service.
- OWASP A08 SSRF — pre-signed URLs are signed by us; ingested image URLs validated against an S3/CDN allow-list.
- OWASP A09 Logging — structured logs to SIEM; PII masked; tamper-evident audit_logs.
- OWASP A02 Crypto — no homegrown crypto; AES-GCM via library; HMAC-SHA-256 for signatures.
- Driver location anti-spoof — server-side speed/jump validation.
- PII masking in logs — phone, email, license, GST, PAN, account_number.

8. Learnings

8.1 Handling Session Tokens & JWT Professionally

JWT works as a stateless authentication token on the client side, carrying user identity and a session ID (sid). Redis acts as a server-side session store to validate whether the session is still active. On each request the server verifies the JWT and cross-checks the sid in Redis. This hybrid approach gives both performance (JWT) and security (session control). Logout removes the session from Redis, invalidating the JWT before its expiration.

8.2 Uploading Images to S3 using Pre-Signed URLs

Pre-signed URLs let clients securely upload directly to S3 without going through the backend. The backend generates a temporary URL with limited permissions, ensuring controlled access. The S3 URL stored in the database is just a reference; access is controlled via private buckets + signed GET URLs or CDN policies. Benefits: faster uploads, lower backend load, secure (no credential exposure), and scalable for large files.

8.3 Why Price Should Not Be Sent from the Frontend

In cart and checkout APIs, the frontend must not send the item price. Frontend data can be modified using browser developer tools, network interceptors, or custom scripts. If the backend trusted such values, users could purchase items at incorrect prices, leading to revenue loss and security issues. Instead, the backend should accept only trusted identifiers (cart_id, item_id, quantity, customisations) and compute the final price server-side from menu_items.price plus customisations, taxes, delivery fees, discounts, and surge pricing. Server-side pricing prevents tampering, centralizes pricing, supports surge/coupons, and follows industry practice in e-commerce and food delivery.

8.4 Why Cart Data Should Be Stored in a Database, Not Only in Memory

In-memory carts are unreliable: a server restart, crash, or load-balanced re-routing loses the cart. Storing carts in a persistent database (e.g., PostgreSQL + Redis cache) ensures the cart survives restarts, remains available across sessions and devices, supports horizontal scaling, and enables analytics, abandoned-cart tracking, and recommendations. Redis is used as a hot read cache only; the source of truth lives in PostgreSQL.

8.5 Why cart_id Should Include User and Device Context

A single user can be logged in from phone, tablet, and laptop simultaneously. If the cart is keyed only by user_id, all devices share one cart and overwrite each other. Including device context (cart = user_id + device_id) prevents conflicts, supports multiple sessions, and enables a better user experience.

8.6 Why Use Base64 + HMAC-SHA-256 in cart_token

External cart tokens use the format cart... Base64 makes the value compact and transport-safe, while the HMAC signature prevents tampering. When the backend receives the token it verifies the HMAC before resolving to the internal UUID cart_id. This stops cart-ID guessing/forgery, prevents access to another user's cart, avoids exposing internal DB IDs, and is a standard pattern for distributed systems.

8.7 State Machines for Orders & Deliveries

Order and delivery transitions are enforced server-side. Invalid status mutations are rejected, providing defense-in-depth against race conditions and replay attacks.

8.8 Idempotency & Distributed Locks

Idempotency keys (UUID per business request) deduplicate at the gateway (Redis) and the database (UNIQUE index). Redis SET NX EX provides distributed locks for assignment and payment paths so a driver is not double-offered and an order is not double-charged.